

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего
образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет инфокоммуникационных технологий

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1**

по дисциплине «Современные методы искусственного интеллекта»

Тема: «Классификация изображений с применением Transfer Learning»

Выполнила:
студентка группы К3320
Чурилина Полина Олеговна

Проверил:
Царёв Д.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Классические методы машинного обучения	5
1.1 Предварительная обработка данных	5
1.2 Кодирование меток.....	6
1.3 Логистический классификатор.....	6
1.4 Наивный байесовский классификатор	9
1.5 Матрица неточностей.....	11
1.6 Машина опорных векторов для прогнозирования дохода	13
1.7 Регрессия	15
1.8 Оценка стоимости недвижимости с помощью SVM-регрессора.	19
2 Классификация изображений с применением Transfer Learning	22
2.1 Выбор датасета	22
2.2 Выбор и обоснование архитектуры	23
2.3 Стратегия Feature Extraction	23
2.4 Стратегия Fine-tuning	26
2.5 Сравнение стратегий	28
ЗАКЛЮЧЕНИЕ	32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	33

ВВЕДЕНИЕ

Цель работы: получить практические навыки распознавания образов, классификации и регрессии данных с помощью обучения с учителем, а также сформировать навыки использования предобученных нейросетевых архитектур (метод transfer learning) для решения задач компьютерного зрения.

Работа состоит из двух частей.

Часть 1 (классические методы машинного обучения, библиотека scikit-learn):

- 1) выполнить предварительную обработку данных (бинаризация, исключение среднего, масштабирование, нормализация);
- 2) выполнить кодирование меток;
- 3) построить логистический классификатор и сравнить результаты при $C = 1$ и $C = 100$;
- 4) построить наивный байесовский классификатор и оценить его метрики с помощью перекрестной проверки;
- 5) построить и проанализировать матрицу неточностей;
- 6) построить классификатор на основе машины опорных векторов (SVM) для прогнозирования уровня дохода;
- 7) построить одномерный, многомерный линейный и полиномиальный регрессоры;
- 8) построить регрессор на основе SVM для оценки стоимости недвижимости.

Часть 2 (transfer learning):

- 1) выбрать датасет для задачи классификации изображений (не менее 3 классов);
- 2) выбрать предобученную архитектуру и обосновать выбор;
- 3) реализовать стратегию Feature Extraction, оценить точность и построить матрицу ошибок;

- 4) реализовать стратегию Fine-tuning с низким темпом обучения;
- 5) сравнить результаты двух подходов по метрикам Accuracy, Precision, Recall и сделать вывод.

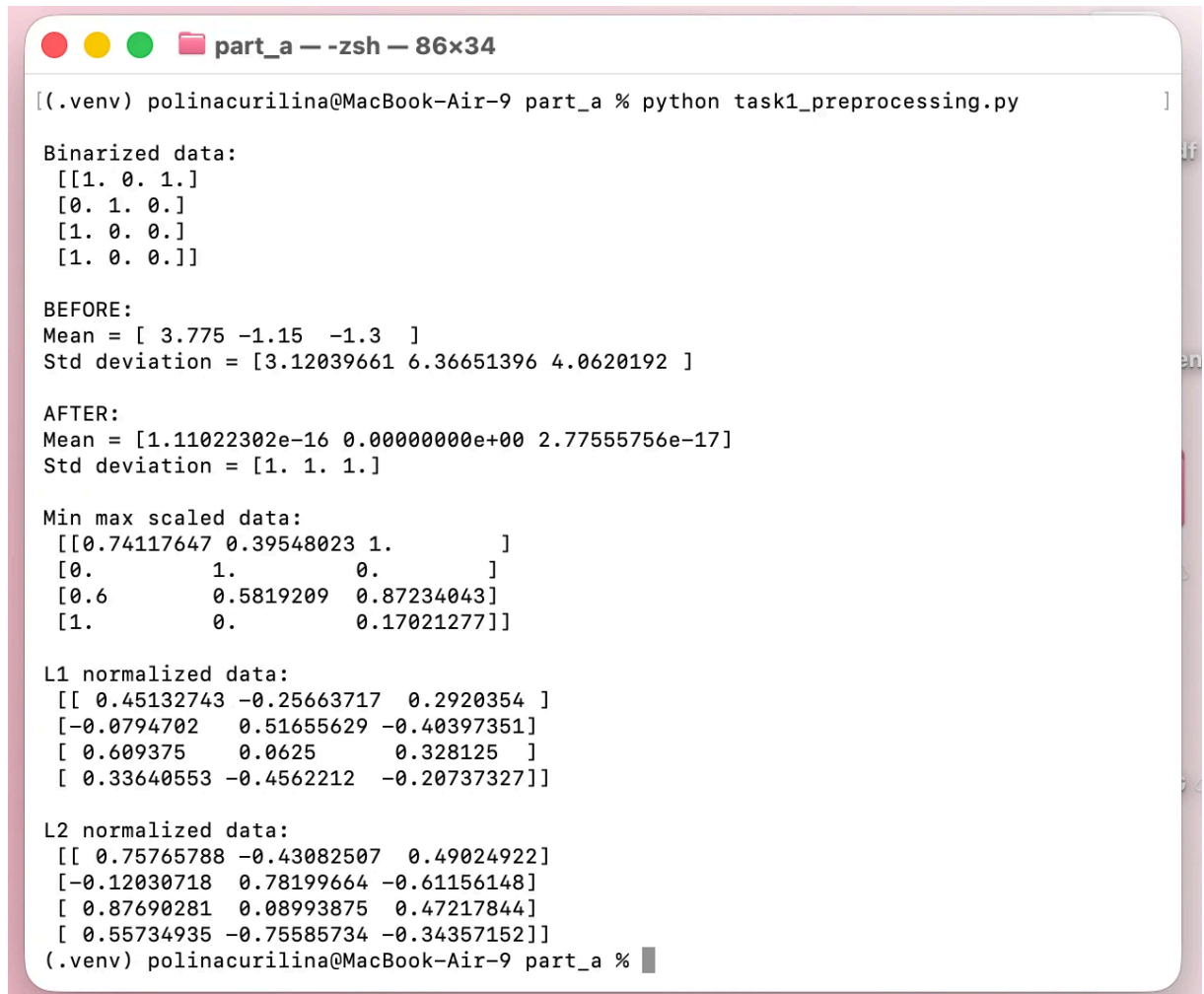
Инструменты: Python 3.12, scikit-learn 1.9, NumPy, Matplotlib, PyTorch 2.14, torchvision 0.29. Исходный код всех программ размещен в репозитории: <https://github.com/churilina-polina/modern-ai-methods/tree/main/lab01>.

Примечание. Код из методических указаний адаптирован под актуальные версии библиотек: модуль `sklearn.cross_validation` заменен на `sklearn.model_selection`, функция `load_boston()` (удалена из `scikit-learn` 1.2) заменена загрузкой того же набора данных с OpenML, а также исправлены ошибки листингов (отсутствующий вызов `fit()` в п. 7.1, повторное создание кодировщика в п. 2).

1 Классические методы машинного обучения

1.1 Предварительная обработка данных

Программа `task1_preprocessing.py` применяет к матрице 4×3 четыре метода предобработки из модуля `sklearn.preprocessing`. Результаты выполнения приведены на рисунке 1.



```
part_a — zsh — 86x34

[ (.venv) polinacurilina@MacBook-Air-9 part_a % python task1_preprocessing.py ]

Binarized data:
[[1. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
 [1. 0. 0.]]

BEFORE:
Mean = [ 3.775 -1.15 -1.3 ]
Std deviation = [3.12039661 6.36651396 4.0620192 ]

AFTER:
Mean = [1.11022302e-16 0.00000000e+00 2.77555756e-17]
Std deviation = [1. 1. 1.]

Min max scaled data:
[[0.74117647 0.39548023 1.
 0. 1. 0.]
 [0.6 0.5819209 0.87234043]
 [1. 0. 0.17021277]]

L1 normalized data:
[[ 0.45132743 -0.25663717 0.2920354 ]
 [-0.0794702 0.51655629 -0.40397351]
 [ 0.609375 0.0625 0.328125 ]
 [ 0.33640553 -0.4562212 -0.20737327]]

L2 normalized data:
[[ 0.75765788 -0.43082507 0.49024922]
 [-0.12030718 0.78199664 -0.61156148]
 [ 0.87690281 0.08993875 0.47217844]
 [ 0.55734935 -0.75585734 -0.34357152]]

(.venv) polinacurilina@MacBook-Air-9 part_a %
```

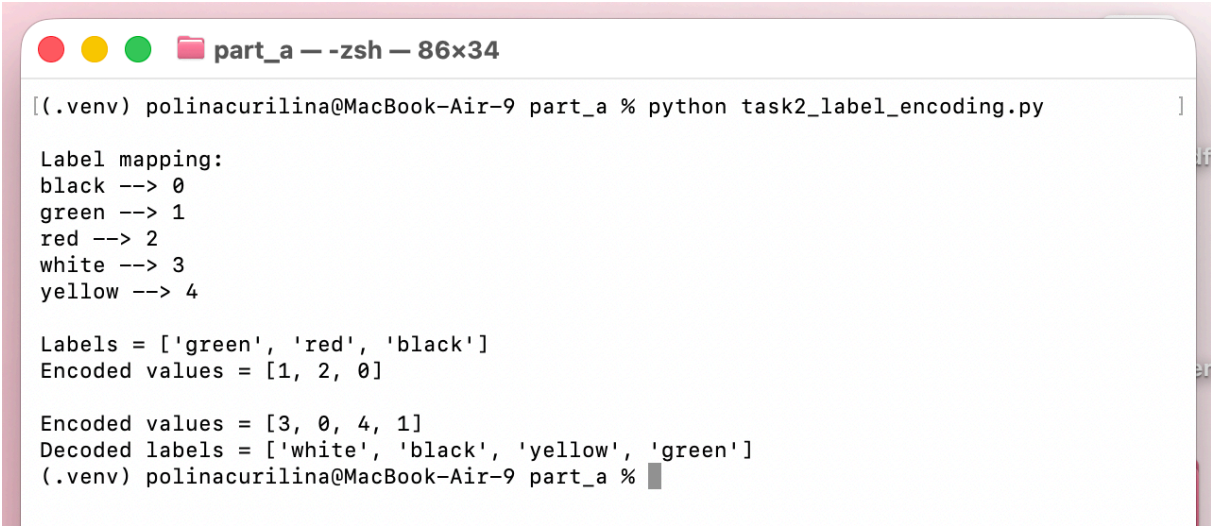
Рисунок 1 — Результаты предварительной обработки данных

- **Бинаризация** с порогом 2.1: значения больше порога стали равны 1, остальные 0.
- **Исключение среднего** (стандартизация): до обработки средние по столбцам были $[3.775; -1.15; -1.3]$, СКО $[3.12; 6.37; 4.06]$; после обработки среднее ≈ 0 (порядка 10^{-16} , погрешность вычислений с плавающей точкой), СКО = 1 для каждого признака.

- **Масштабирование MinMax**: каждый столбец приведен к диапазону [0; 1], минимальное значение признака стало 0, максимальное 1.
- **Нормализация**: при L1 сумма модулей в каждой строке равна 1, при L2 сумма квадратов равна 1. Нормализация применяется к строкам (объектам), а масштабирование к столбцам (признакам).

1.2 Кодирование меток

Программа `task2_label_encoding.py` обучает `LabelEncoder` на наборе цветов и выполняет прямое и обратное преобразование (рисунок 2).



```

[part_a] polinacurilina@MacBook-Air-9 part_a % python task2_label_encoding.py

Label mapping:
black --> 0
green --> 1
red --> 2
white --> 3
yellow --> 4

Labels = ['green', 'red', 'black']
Encoded values = [1, 2, 0]

Encoded values = [3, 0, 4, 1]
Decoded labels = ['white', 'black', 'yellow', 'green']
[part_a] polinacurilina@MacBook-Air-9 part_a %

```

Рисунок 2 — Результат кодирования и декодирования меток

Кодировщик сортирует уникальные метки по алфавиту и присваивает им номера: `black` $\rightarrow$ 0, `green` $\rightarrow$ 1, `red` $\rightarrow$ 2, `white` $\rightarrow$ 3, `yellow` $\rightarrow$ 4. Метки `[green, red, black]` закодированы как `[1, 2, 0]`, набор чисел `[3, 0, 4, 1]` декодирован обратно в `[white, black, yellow, green]`, то есть преобразование обратимо. Недостаток такого кодирования в том, что он вводит искусственный порядок между категориями; для линейных моделей в таком случае лучше использовать one-hot кодирование.

1.3 Логистический классификатор

Программа `task3_logistic_regression.py` обучает логистическую регрессию на 12 двумерных точках четырех классов. Значение `C` передается

аргументом командной строки. Результаты для $C = 1$ и $C = 100$ приведены на рисунках 3 и 4.

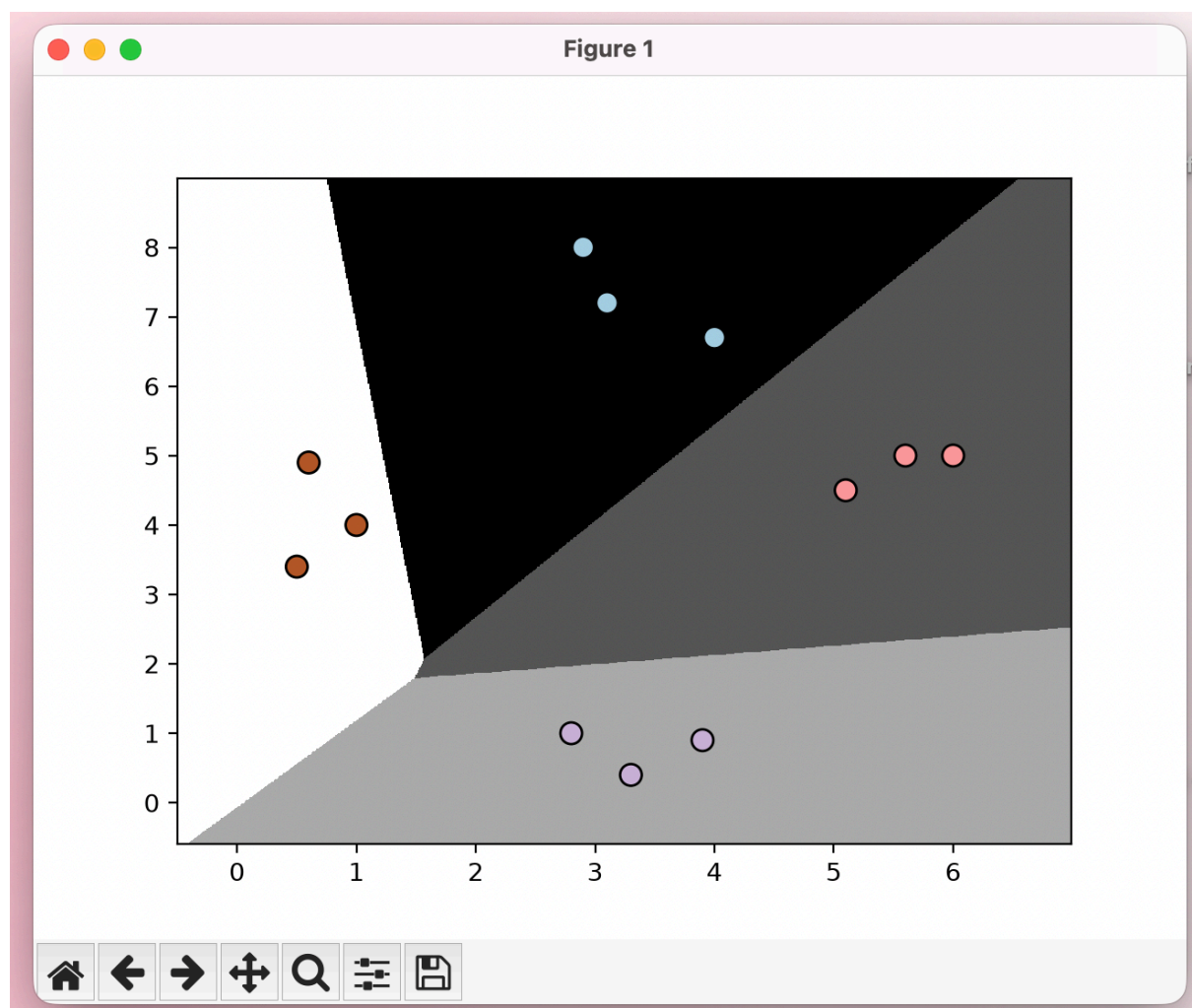


Рисунок 3 — Границы классов логистического классификатора, $C = 1$

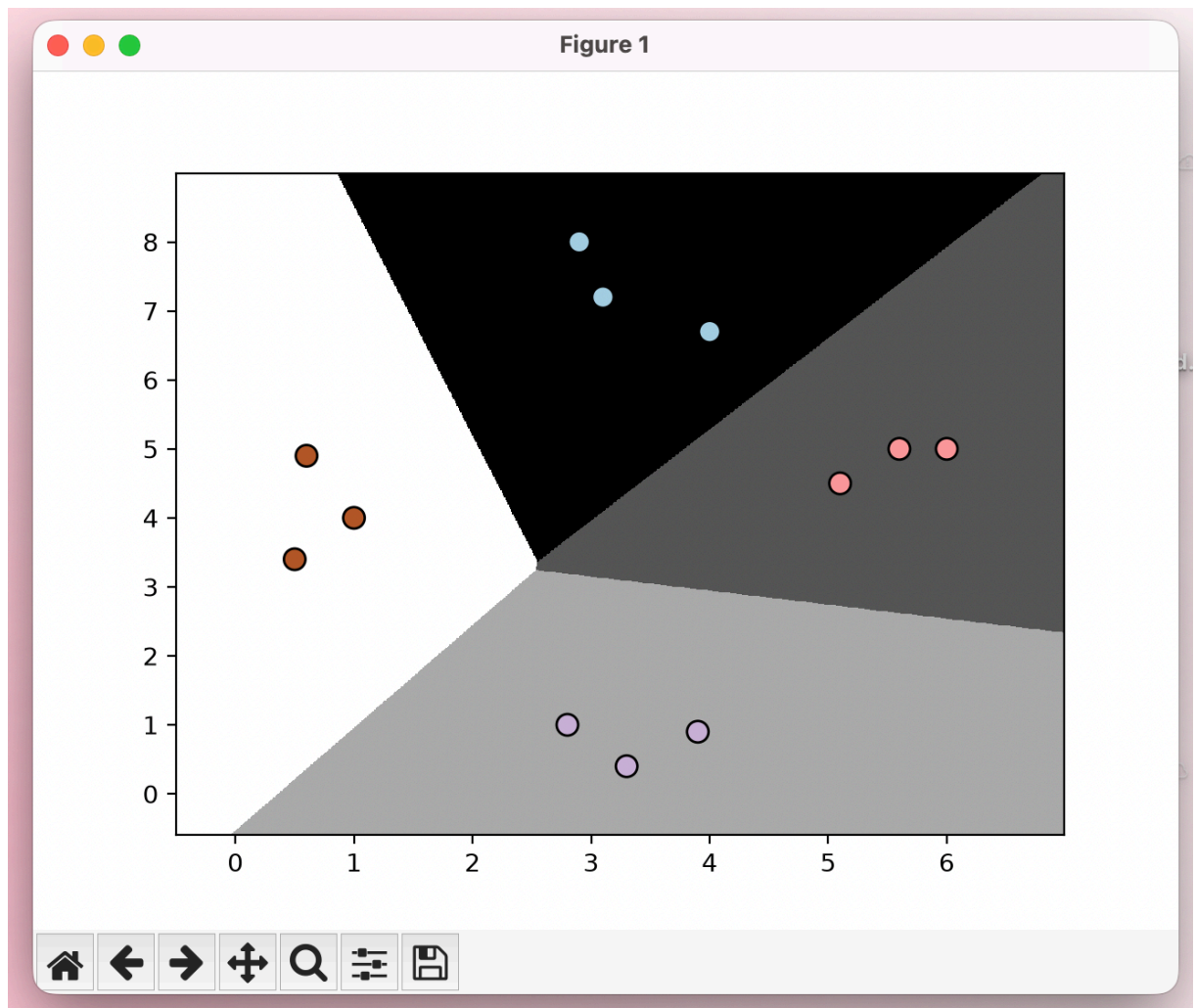


Рисунок 4 — Границы классов логистического классификатора, $C = 100$

Анализ. Параметр C обратен силе L2-регуляризации: функция потерь имеет вид $\|w\|^2/2 + C \cdot \sum \log\text{-loss}$. При обоих значениях все 12 обучающих точек классифицированы верно (ассигура = 100 %), но границы классов заметно отличаются.

- При $C = 1$ регуляризация сильная, веса модели малы (норма векторов весов 0.5–1.3), поэтому модель «неуверенна»: максимальная вероятность класса для обучающих точек всего 0.45–0.68. Границы проходят несимметрично: точка схождения областей смещена к точкам класса 3 (белая область слева), граница почти касается их.
- При $C = 100$ регуляризация слабая, штраф за ошибки на обучающих данных больше, веса выросли до 2.1–4.8, а уверенность до 0.84–0.99. Границы стали более «честными»: точка схождения сместилась к центру между четырьмя группами ($\approx (2.5; 3.3)$), каждая граница

проходит примерно посередине между соседними кластерами, то есть зазор до точек увеличился.

Вывод: увеличение C позволяет модели точнее подстроиться под обучающие данные. На маленькой и чисто разделимой выборке это полезно, но на зашумленных данных большое C ведет к переобучению, а малое к недообучению. Оптимальное значение подбирается перекрестной проверкой.

1.4 Наивный байесовский классификатор

Программа `task4_naive_bayes.py` обучает гауссовский наивный байесовский классификатор `GaussianNB` на файле `data_multivar_nb.txt` (400 точек, 4 класса). Разбиение данных на всей выборке и на тестовой выборке показано на рисунках 5 и 6, метрики на рисунке 7.

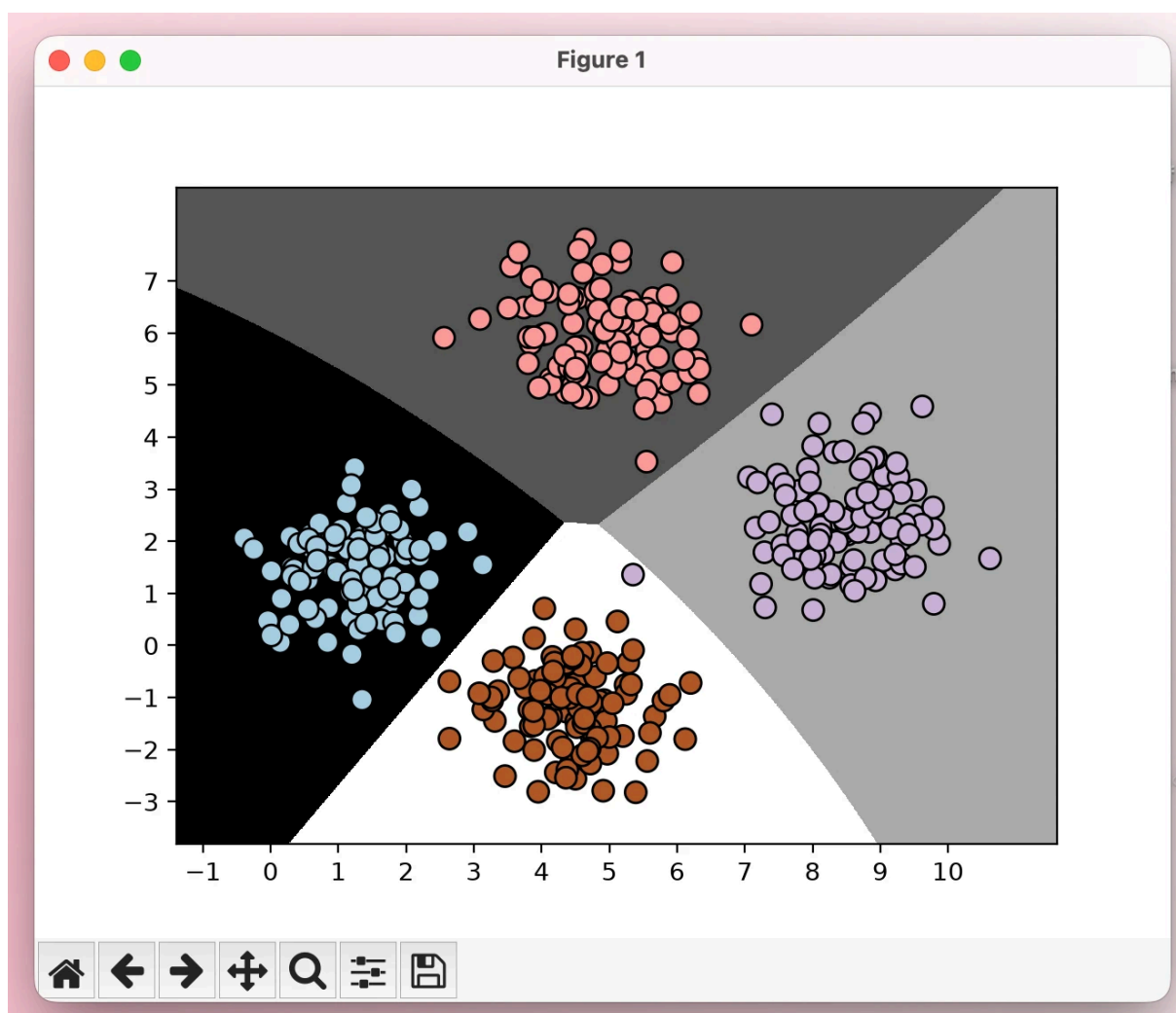


Рисунок 5 — Разбиение данных наивным байесовским классификатором (обучение на всей выборке)

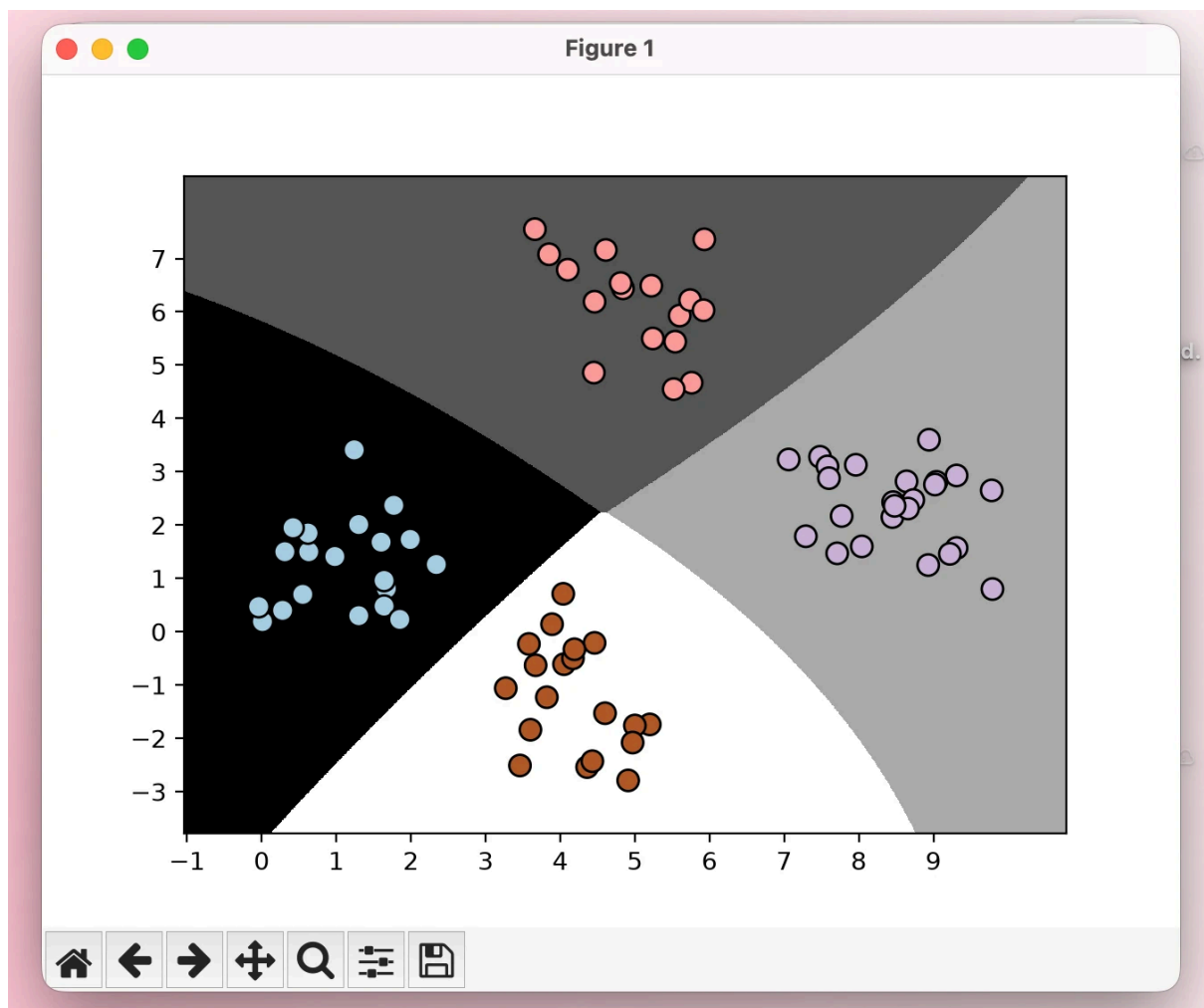


Рисунок 6 — Разбиение тестовой выборки классификатором, обученным на 80 % данных

```
part_a — -zsh — 86x47
[(.venv) polinacurilina@MacBook-Air-9 part_a % python task4_naive_bayes.py
Accuracy of Naive Bayes classifier = 99.75 %
Accuracy of the new classifier = 100.0 %
Accuracy: 99.75%
Precision: 99.76%
Recall: 99.75%
F1: 99.75%
(.venv) polinacurilina@MacBook-Air-9 part_a % ]
```

Рисунок 7 — Показатели качества наивного байесовского классификатора

Таблица 1 — Показатели наивного байесовского классификатора

Способ оценки	Значение
Ассурасу на обучающих данных	99.75 %
Ассурасу на отложенной выборке 20 %	100.0 %
Accuracy, 3-fold CV	99.75 %
Precision (weighted), 3-fold CV	99.76 %
Recall (weighted), 3-fold CV	99.75 %
F1 (weighted), 3-fold CV	99.75 %

Анализ. Классы образуют четыре компактных, хорошо разделенных облака, поэтому предположение о нормальном распределении признаков внутри класса выполняется, и классификатор ошибается только на одной точке из 400 (она лежит на границе двух облаков). Границы классов криволинейные (квадратичные), так как у каждого класса своя дисперсия. Оценка на обучающих данных ненадежна, так как модель видела эти точки; результат на отложенной выборке (100 % на 80 точках) и перекрестная проверка (99.75 %) подтверждают, что модель не переобучена. Близость precision, recall и F1 друг к другу говорит о том, что ошибок нет ни в одном из классов в заметном количестве.

1.5 Матрица неточностей

Программа `task5_confusion_matrix.py` строит матрицу неточностей и отчет о классификации для 11 заданных пар «истинная/предсказанная метка» (рисунки 8 и 9).

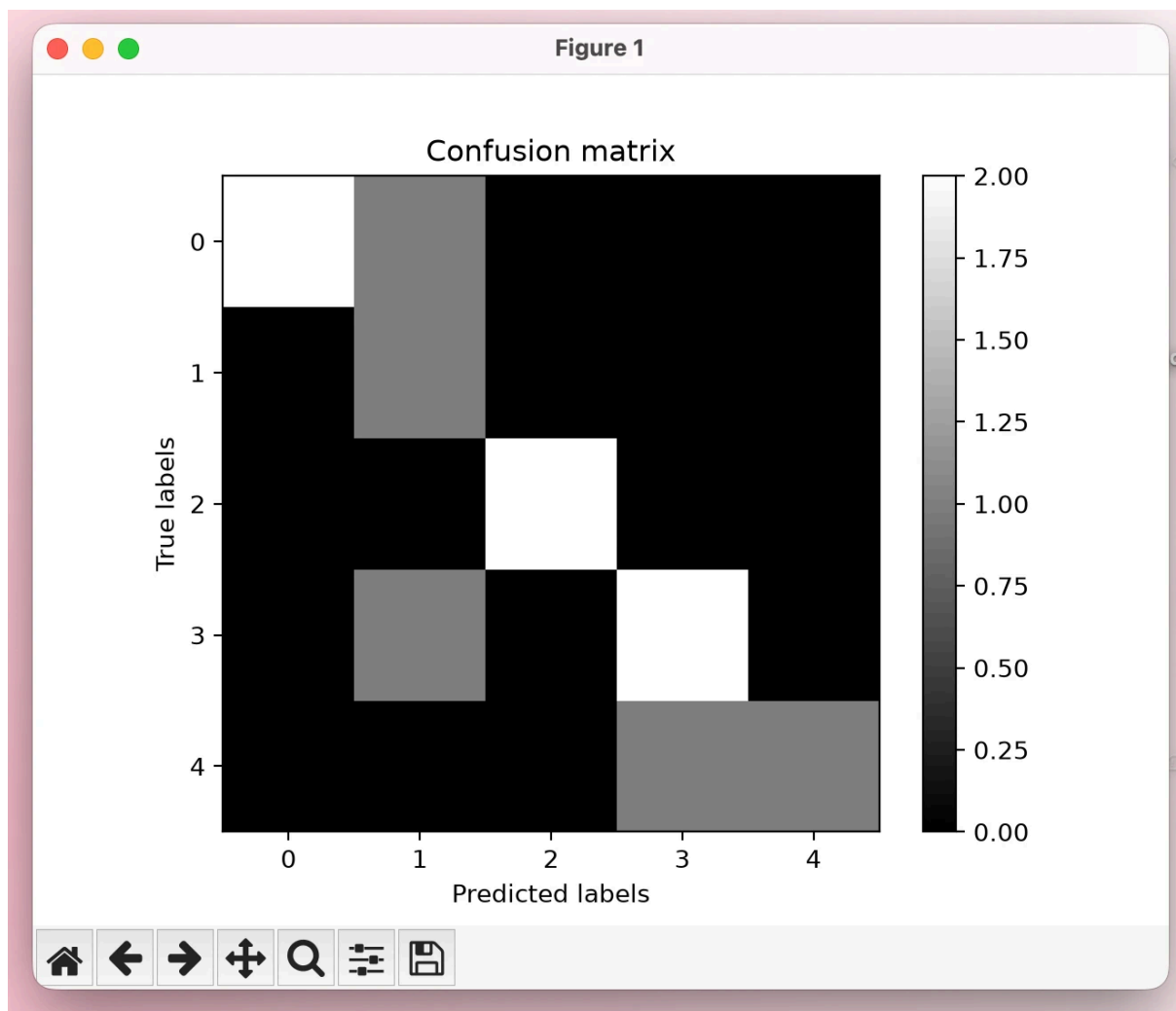


Рисунок 8 — Матрица неточностей

```
part_a — Python task5_confusion_matrix.py — 86x34

[(.venv) polinacurilina@MacBook-Air-9 part_a % python task5_confusion_matrix.py ]
Confusion matrix:
[[2 1 0 0 0]
 [0 1 0 0 0]
 [0 0 2 0 0]
 [0 1 0 2 0]
 [0 0 0 1 1]]

precision    recall  f1-score   support

Class-0      1.00    0.67    0.80         3
Class-1      0.33    1.00    0.50         1
Class-2      1.00    1.00    1.00         2
Class-3      0.67    0.67    0.67         3
Class-4      1.00    0.50    0.67         2

accuracy          0.73         11
macro avg         0.80    0.77    0.73         11
weighted avg      0.85    0.73    0.75         11
```

Рисунок 9 — Отчет о результатах классификации

Анализ. Верно классифицированы 8 из 11 объектов (accuracy = 0.73), они находятся на главной диагонали. Ошибки: один объект класса 0 отнесен к классу 1, один объект класса 3 к классу 1, один объект класса 4 к классу 3.

- Класс 2 распознан идеально (precision = recall = 1.00).
- Класс 1 имеет recall = 1.00, но precision всего 0.33: единственный объект класса 1 найден, но к нему же ошибочно отнесены два чужих объекта (ложноположительные срабатывания).
- Классы 0 и 4 имеют precision = 1.00, но пониженный recall (0.67 и 0.50): часть их объектов пропущена (ложноотрицательные).
- Macro-усреднение $F1 = 0.73$, weighted $F1 = 0.75$: взвешенное среднее выше, так как лучше распознаются более многочисленные классы.

Матрица неточностей показывает не только долю ошибок, но и то, какие именно классы путаются, что нельзя увидеть по одной метрике accuracy.

1.6 Машина опорных векторов для прогнозирования дохода

Программа `task6_svm_income.py` решает задачу бинарной классификации (доход $\leq 50K$ или $> 50K$) по 14 признакам набора Adult Census Income. Строковые признаки кодируются отдельными `LabelEncoder`, числовые используются без изменений. Результаты приведены на рисунке 10 и в таблицах 2, 3.

```

part_a — -zsh — 86x34

[(.venv) polinacurilina@MacBook-Air-9 part_a % python task6_svm_income.py
F1 score: 76.09%

--- Предсказания LinearSVC для тестовых точек ---
37 HS-grad Handlers-cleaners 40h/week -> <=50K
52 Masters Exec-managerial 60h/week -> >50K
23 Some-college Sales 25h/week -> <=50K
45 Doctorate Prof-specialty 50h/week -> >50K

--- Сравнение ядер и параметров (на тестовой выборке 20%) ---
Модель                Accuracy Precision Recall F1
LinearSVC, без масштабирования 79.56% 79.26% 79.56% 75.75%
LinearSVC, со StandardScaler 81.70% 80.76% 81.70% 79.93%
SVC linear, C=1 81.88% 80.99% 81.88% 80.14%
SVC rbf, C=1 83.11% 82.28% 83.11% 82.26%
SVC rbf, C=10 82.89% 82.18% 82.89% 82.35%
SVC poly (deg=3), C=1 82.18% 81.27% 82.18% 80.67%
SVC sigmoid, C=1 75.57% 75.38% 75.57% 75.47%

--- Предсказания SVC(rbf, C=10) для тестовых точек ---
37 HS-grad Handlers-cleaners 40h/week -> <=50K
52 Masters Exec-managerial 60h/week -> >50K
23 Some-college Sales 25h/week -> <=50K
45 Doctorate Prof-specialty 50h/week -> >50K
(.venv) polinacurilina@MacBook-Air-9 part_a %

```

Рисунок 10 — Результаты SVM-классификатора уровня дохода

Для классификатора из методических указаний (OneVsOneClassifier(LinearSVC)) F1-мера по трехкратной перекрестной проверке составила 76.09 % (в методичке 66.82 %: отличие связано с изменением реализации LinearSVC в новых версиях scikit-learn). Далее были сравнены разные ядра и параметры (таблица 2). Ядерные SVC обучались на подвыборке из 8000 объектов, так как время их обучения растет квадратично от объема данных.

Таблица 2 — Сравнение ядер SVM (тестовая выборка 20 %, weighted-усреднение)

Модель	Accuracy	Precision	Recall	F1
LinearSVC без масштабирования	79.56 %	79.26 %	79.56 %	75.75 %
LinearSVC + StandardScaler	81.70 %	80.76 %	81.70 %	79.93 %
SVC linear, C = 1	81.88 %	80.99 %	81.88 %	80.14 %
SVC rbf, C = 1	83.11 %	82.28 %	83.11 %	82.26 %
SVC rbf, C = 10	82.89 %	82.18 %	82.89 %	82.35 %
SVC poly (степень 3), C = 1	82.18 %	81.27 %	82.18 %	80.67 %
SVC sigmoid, C = 1	75.57 %	75.38 %	75.57 %	75.47 %

Таблица 3 — Предсказания для тестовых точек данных

Возраст, образование, должность, ч/нед	LinearSVC	SVC rbf
37, HS-grad, Handlers-cleaners, 40	$\leq 50K$	$\leq 50K$
52, Masters, Exec-managerial, 60	$> 50K$	$> 50K$
23, Some-college, Sales, 25	$\leq 50K$	$\leq 50K$
45, Doctorate, Prof-specialty, 50	$> 50K$	$> 50K$

Анализ. Масштабирование признаков заметно улучшает линейный SVM (F1 с 75.75 % до 79.93 %): без него признаки с большими значениями ($\text{fnlwgt} \sim 10^5$, $\text{capital-gain} \sim 10^4$) доминируют в расстоянии, а оптимизация плохо сходится. Лучший результат показало RBF-ядро ($F1 \approx 82.3\%$), так как зависимость дохода от признаков нелинейна. Сигмоидальное ядро оказалось худшим. Precision во всех случаях немного ниже ассигасы, так как классы несбалансированы (около 75 % объектов с доходом $\leq 50K$), и модель чаще ошибается на меньшем классе. Предсказания для всех четырех точек логичны: молодой человек с неполной занятостью и рабочий без высшего образования отнесены к классу $\leq 50K$, руководитель с магистерской степенью и специалист с докторской степенью, работающие 50–60 часов в неделю, к классу $> 50K$.

1.7 Регрессия

1.7.1 Регрессор одной переменной

Программа `task7_1_single_var_regression.py` обучает линейную регрессию на файле `data_singlevar_regr.txt` (60 точек, 80 % обучение, 20 % тест), сохраняет модель в файл `model.pkl` с помощью `pickle` и загружает ее обратно (рисунки 11, 12).

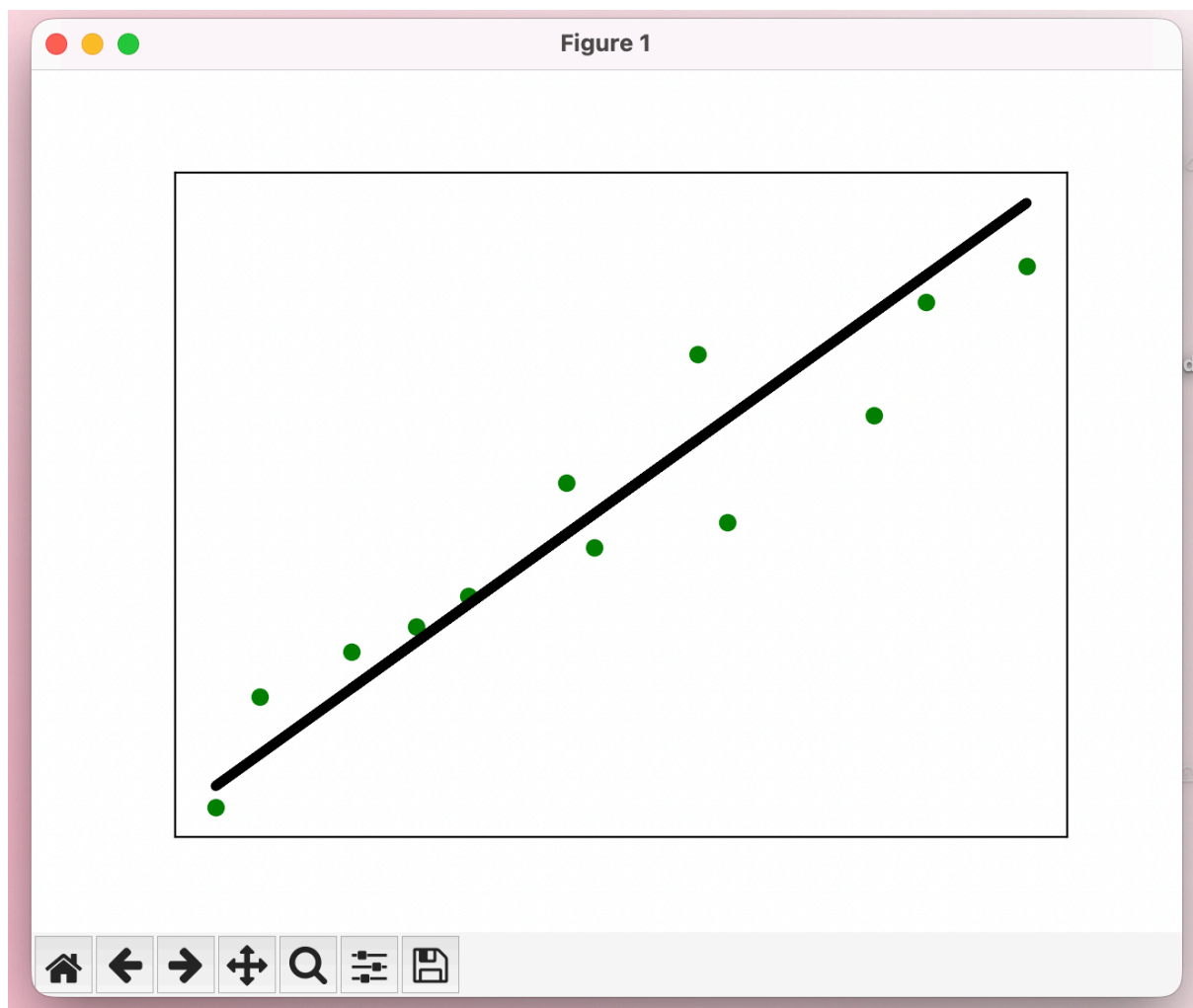


Рисунок 11 — Результат предсказания одномерного линейного регрессора

```
part_a — Python task7_1_single_var_regression.py — 86x34

[ (.venv) polinacurilina@MacBook-Air-9 part_a % python task7_1_single_var_regression.py ]
Linear regressor performance:
Mean absolute error = 0.59
Mean squared error = 0.49
Median absolute error = 0.51
Explain variance score = 0.86
R2 score = 0.86

New mean absolute error = 0.59
```

Рисунок 12 — Метрики одномерного регрессора

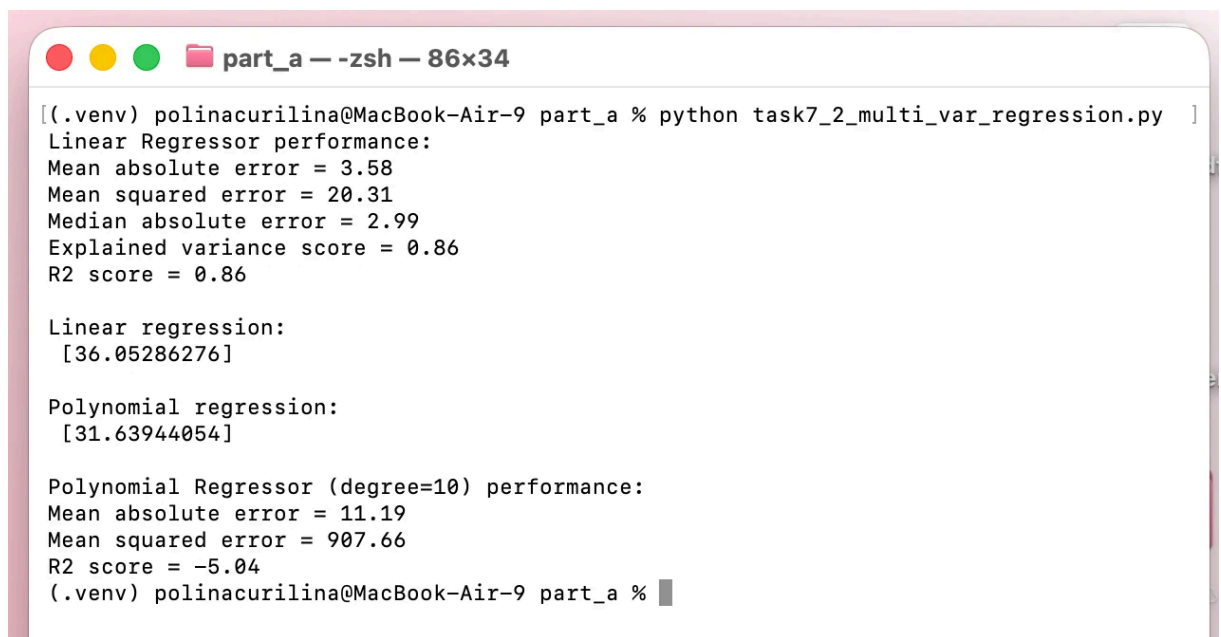
Таблица 4 — Метрики одномерного линейного регрессора

Метрика	Значение
Mean absolute error (MAE)	0.59
Mean squared error (MSE)	0.49
Median absolute error	0.51
Explained variance score	0.86
R^2	0.86
MAE модели, загруженной из model.pkl	0.59

Анализ. Коэффициент детерминации $R^2 = 0.86$: модель объясняет 86 % дисперсии целевой переменной, то есть зависимость близка к линейной. Средняя абсолютная ошибка 0.59 при разбросе значений у примерно от 1 до 7 невелика. Медианная ошибка (0.51) меньше средней, значит, крупных выбросов немного. Совпадение explained variance и R^2 говорит о том, что систематического смещения предсказаний нет. Ошибка загруженной модели совпадает с исходной, значит, сериализация через pickle сохраняет модель без потерь.

1.7.2 Многомерный регрессор

Программа `task7_2_multi_var_regression.py` обучает линейный и полиномиальный (степень 10) регрессоры на файле `data_multivar_regr.txt` (700 точек, 3 признака) и сравнивает их прогнозы для точки `[7.75, 6.35, 5.56]` (рисунок 13).



```
part_a — -zsh — 86x34

[ (.venv) polinacurilina@MacBook-Air-9 part_a % python task7_2_multi_var_regression.py ]
Linear Regressor performance:
Mean absolute error = 3.58
Mean squared error = 20.31
Median absolute error = 2.99
Explained variance score = 0.86
R2 score = 0.86

Linear regression:
[36.05286276]

Polynomial regression:
[31.63944054]

Polynomial Regressor (degree=10) performance:
Mean absolute error = 11.19
Mean squared error = 907.66
R2 score = -5.04
(.venv) polinacurilina@MacBook-Air-9 part_a %
```

Рисунок 13 — Результаты многомерного линейного и полиномиального регрессоров

Таблица 5 — Сравнение многомерных регрессоров

Показатель	Линейный	Полиномиальный (степень 10)
Прогноз для [7.75, 6.35, 5.56] (ождается ≈ 41.35)	36.05	31.64
MAE на тестовой выборке	3.58	11.19
MSE на тестовой выборке	20.31	907.66
R^2 на тестовой выборке	0.86	-5.04

Анализ. Линейный регрессор показывает хорошее качество на тестовой выборке ($R^2 = 0.86$, MAE = 3.58). Для выбранной точки он дал прогноз 36.05 при ожидаемом значении около 41.35 (близкая точка [7.66, 6.29, 5.66] из строки 11 файла данных), то есть ошибся примерно на 5.3.

Полиномиальный регрессор 10-й степени, вопреки ожиданию из методических указаний, дал для этой точки прогноз 31.64: он ошибся сильнее линейного (примерно на 9.7). Проверка на всей тестовой выборке подтверждает, что полиномиальная модель хуже: MAE = 11.19 против 3.58, MSE = 907.66 против 20.31, а $R^2 = -5.04$, то есть модель предсказывает хуже, чем простое среднее значение. Причины:

- **переобучение:** полином 10-й степени от трех переменных порождает 286 признаков при 560 обучающих точках, и модель подгоняется под шум обучающей выборки;
- **плохая обусловленность:** старшие признаки достигают значений порядка $7.75^{10} \approx 8 \cdot 10^8$, а младшие порядка 1, поэтому задача наименьших квадратов численно неустойчива. Это проявилось и практически: при запуске того же кода в scikit-learn версии 1.8 прогноз для точки составил 41.08 при $R^2 = -588$ на тесте, а в версии 1.9 получилось 31.64 и $R^2 = -5.04$, то есть результат зависит даже от деталей реализации решателя.

Вывод: высокая степень полинома не гарантирует точности, и оценивать модель по одной точке нельзя. Для полиномиальной регрессии нужно брать небольшую степень (2–3), масштабировать признаки и

использовать регуляризацию (Ridge, Lasso), а выбирать степень по ошибке на тестовой выборке или перекрестной проверке.

1.8 Оценка стоимости недвижимости с помощью SVM-регрессора

Программа `task8_svr_housing.py` обучает SVR с линейным ядром на наборе Boston Housing (506 домов, 13 признаков) при разных значениях C и ϵ и прогнозирует цену для четырех точек: точки из методических указаний и трех реальных домов из набора (для них известна фактическая цена, тыс. \$). Результаты приведены на рисунке 14 и в таблице 6.

```

part_a — zsh — 86x47

[(.venv) polinacurilina@MacBook-Air-9 part_a % python task8_svr_housing.py

#### Performance: C=1.0, epsilon=0.1 ####
Mean squared error = 15.92
Explained variance score = 0.81
Number of support vectors = 397
Predicted price [Точка из методички]: 23.43
Predicted price [Дом №0 из датасета]: 27.11 (реальная цена: 24.0)
Predicted price [Дом №283 из датасета]: 42.16 (реальная цена: 50.0)
Predicted price [Дом №400 из датасета]: 12.80 (реальная цена: 5.6)

#### Performance: C=10.0, epsilon=0.1 ####
Mean squared error = 15.84
Explained variance score = 0.81
Number of support vectors = 402
Predicted price [Точка из методички]: 23.20
Predicted price [Дом №0 из датасета]: 27.50 (реальная цена: 24.0)
Predicted price [Дом №283 из датасета]: 43.25 (реальная цена: 50.0)
Predicted price [Дом №400 из датасета]: 12.82 (реальная цена: 5.6)

#### Performance: C=0.1, epsilon=0.1 ####
Mean squared error = 17.19
Explained variance score = 0.8
Number of support vectors = 397
Predicted price [Точка из методички]: 23.28
Predicted price [Дом №0 из датасета]: 28.06 (реальная цена: 24.0)
Predicted price [Дом №283 из датасета]: 39.12 (реальная цена: 50.0)
Predicted price [Дом №400 из датасета]: 11.41 (реальная цена: 5.6)

#### Performance: C=1.0, epsilon=1.0 ####
Mean squared error = 15.51
Explained variance score = 0.81
Number of support vectors = 297
Predicted price [Точка из методички]: 22.61
Predicted price [Дом №0 из датасета]: 27.91 (реальная цена: 24.0)
Predicted price [Дом №283 из датасета]: 42.48 (реальная цена: 50.0)
Predicted price [Дом №400 из датасета]: 12.33 (реальная цена: 5.6)

#### Performance: C=1.0, epsilon=5.0 ####
Mean squared error = 15.82
Explained variance score = 0.81
Number of support vectors = 91
Predicted price [Точка из методички]: 26.50
Predicted price [Дом №0 из датасета]: 29.76 (реальная цена: 24.0)
Predicted price [Дом №283 из датасета]: 43.67 (реальная цена: 50.0)
Predicted price [Дом №400 из датасета]: 11.71 (реальная цена: 5.6)
(.venv) polinacurilina@MacBook-Air-9 part_a %

```

Рисунок 14 — Результаты SVM-регрессора для разных C и epsilon

Таблица 6 — Результаты SVR (линейное ядро) при разных параметрах

C	ϵ	MSE	EVS	Опорных векторов	Точка из методички	Дом №0 (24.0)	Дом №283 (50.0)	Дом №400 (5.6)
1	0.1	15.92	0.81	397	23.43	27.11	42.16	12.80
10	0.1	15.84	0.81	402	23.20	27.50	43.25	12.82
0.1	0.1	17.19	0.80	397	23.28	28.06	39.12	11.41
1	1.0	15.51	0.81	297	22.61	27.91	42.48	12.33
1	5.0	15.82	0.81	91	26.50	29.76	43.67	11.71

Анализ. Параметр C задает штраф за ошибки: при $C = 0.1$ модель сильно регуляризована и недообучена (MSE выросла до 17.19, прогноз для дорогого дома №283 занижен до 39.1 при реальных 50.0). Увеличение C до 10 почти не меняет качество (MSE 15.84): для линейной модели дальнейшее уменьшение регуляризации не дает выигрыша. Параметр ϵ задает ширину «трубки», внутри которой ошибки не штрафуются: с ростом ϵ от 0.1 до 5 число опорных векторов упало с 397 до 91, то есть модель стала гораздо компактнее при почти том же MSE; при $\epsilon = 1$ получен лучший MSE (15.51). Слишком большое ϵ (5 тыс. \$) делает модель грубее: прогнозы смещаются (для точки из методички 26.5 вместо ≈ 23). Во всех конфигурациях модель переоценивает дешевое жилье (дом №400: 11–13 вместо 5.6) и недооценивает дорогое (дом №283: 39–44 вместо 50), что характерно для линейной модели: крайние значения цены зависят от признаков нелинейно, а цена 50.0 в этом наборе к тому же искусственно ограничена сверху. Explained variance около 0.81 во всех случаях: линейный SVR объясняет примерно 81 % разброса цен.

2 Классификация изображений с применением Transfer Learning

2.1 Выбор датасета

Для работы выбран открытый набор данных Oxford-IIIT Pet [5]: около 7 400 цветных фотографий кошек и собак 37 пород (12 пород кошек и 25 пород собак), примерно по 200 изображений на класс. Задача является задачей мелкозернистой (fine-grained) классификации: многие породы похожи друг на друга, а фотографии сделаны в разных условиях (фон, ракурс, освещение, позы животных).

Использовано официальное разбиение: выборка trainval (3 680 изображений) разделена на обучающую (3 312) и валидационную (368) части в соотношении 90/10, выборка test (3 669 изображений) использована только для итоговой оценки. Изображения приведены к размеру 224×224 и нормализованы по статистикам ImageNet. Для обучающей выборки применена аугментация: случайная обрезка с масштабированием, горизонтальное отражение, изменение яркости, контраста и насыщенности. Примеры изображений приведены на рисунке 15.



Рисунок 15 — Примеры изображений набора Oxford-IIIT Pet

2.2 Выбор и обоснование архитектуры

В качестве базовой сети выбрана MobileNetV2 [6], предобученная на ImageNet (веса IMAGENET1K_V2). Обоснование выбора:

- малое число параметров (3.5 млн против 25.6 млн у ResNet-50 и 5.3 млн у EfficientNet-B0) и вычислительная сложность около 0.3 GFLOPs, что позволяет обучать модель на ноутбуке без дискретной видеокарты;
- достаточно высокая точность на ImageNet ($\text{top-1} \approx 72\%$), обеспечивающая качественные универсальные признаки;
- ImageNet содержит более 100 классов пород собак и несколько классов кошек, поэтому признаки, выученные сетью, близки к целевой задаче, что делает перенос обучения особенно эффективным.

Архитектура MobileNetV2 построена из inverted residual блоков. Каждый блок сначала расширяет число каналов сверткой 1×1 (обычно в 6 раз), затем применяет depthwise-свертку 3×3 (каждый канал обрабатывается отдельно) и снова сжимает каналы сверткой 1×1 без функции активации (linear bottleneck). Если размерности совпадают, вход блока складывается с выходом (residual connection). Разделение обычной свертки на depthwise и pointwise уменьшает объем вычислений примерно в 8–9 раз. Сеть содержит 19 блоков в модуле features, после которых идут глобальный average pooling (вектор признаков размерности 1280) и полносвязный классификатор.

Для переноса обучения последний слой $\text{Linear}(1280 \rightarrow 1000)$ заменен на $\text{Linear}(1280 \rightarrow 37)$ под число пород.

2.3 Стратегия Feature Extraction

Все веса сверточной части (features) заморожены (`requires_grad = False`), и сеть используется как фиксированный извлекатель признаков. Обучается только новый выходной слой: 47 397 параметров из 2 271 269 (2.09 %). Оптимизатор Adam, learning rate = 10^{-3} , 6 эпох, batch size = 32, функция потерь CrossEntropyLoss. Сохраняются веса эпохи с лучшей точностью на

валидации. Ход обучения, метрики и матрица ошибок приведены на рисунках 16–18.

3. Стратегия 1: Feature Extraction

Замораживаем **все** веса сверточной части (`requires_grad = False`), так что она работает как фиксированный извлекатель признаков (вектор 1280 чисел). Обучаем только новый слой `Linear(1280 → 37)`.

```
[6]: fe_model = build_model()
     for p in fe_model.features.parameters():
         p.requires_grad = False

     total, trainable = count_params(fe_model)
     print(f'Обучаемых параметров: {trainable:,} из {total:,} ({100*trainable/total:.2f}%)')

     EPOCHS_FE = 6
     opt_fe = torch.optim.Adam(fe_model.classifier.parameters(), lr=1e-3)
     t0 = time.time()
     hist_fe = fit(fe_model, opt_fe, EPOCHS_FE, tag='FE')
     time_fe = time.time() - t0
     print(f'Время обучения: {time_fe/60:.1f} мин')

Обучаемых параметров: 47,397 из 2,271,269 (2.09%)
[FE] epoch 1/6  train loss 2.190 acc 0.582 | val loss 1.185 acc 0.837 | 16s
[FE] epoch 2/6  train loss 0.982 acc 0.833 | val loss 0.742 acc 0.867 | 15s
[FE] epoch 3/6  train loss 0.686 acc 0.867 | val loss 0.606 acc 0.867 | 15s
[FE] epoch 4/6  train loss 0.545 acc 0.889 | val loss 0.519 acc 0.878 | 15s
[FE] epoch 5/6  train loss 0.462 acc 0.900 | val loss 0.473 acc 0.864 | 15s
[FE] epoch 6/6  train loss 0.414 acc 0.906 | val loss 0.437 acc 0.875 | 16s
Время обучения: 1.5 мин
```

Рисунок 16 — Процесс обучения в режиме Feature Extraction

```
[7]: y_true, y_pred_fe = predict(fe_model, test_loader)
     m_fe = metrics(y_true, y_pred_fe)
     print('Feature Extraction, метрики на тестовой выборке:')
     for k, v in m_fe.items(): print(f' {k:10s} = {100*v:.2f}%',)
     print('\nСамые частые ошибки:'); top_confusions(y_true, y_pred_fe)

Feature Extraction, метрики на тестовой выборке:
Accuracy   = 85.12%
Precision   = 85.33%
Recall      = 85.04%
F1          = 84.85%

Самые частые ошибки:
American Pit Bull Terrier  -> Staffordshire Bull Terrier : 24 ошибок
Bengal                     -> Egyptian Mau             : 23 ошибок
Birman                    -> Ragdoll                 : 18 ошибок
Staffordshire Bull Terrier -> American Pit Bull Terrier : 16 ошибок
British Shorthair         -> Russian Blue          : 16 ошибок

[8]: plot_confusion(y_true, y_pred_fe, 'Confusion matrix: Feature Extraction (MobileNetV2)')
```

Рисунок 17 — Метрики модели Feature Extraction на тестовой выборке

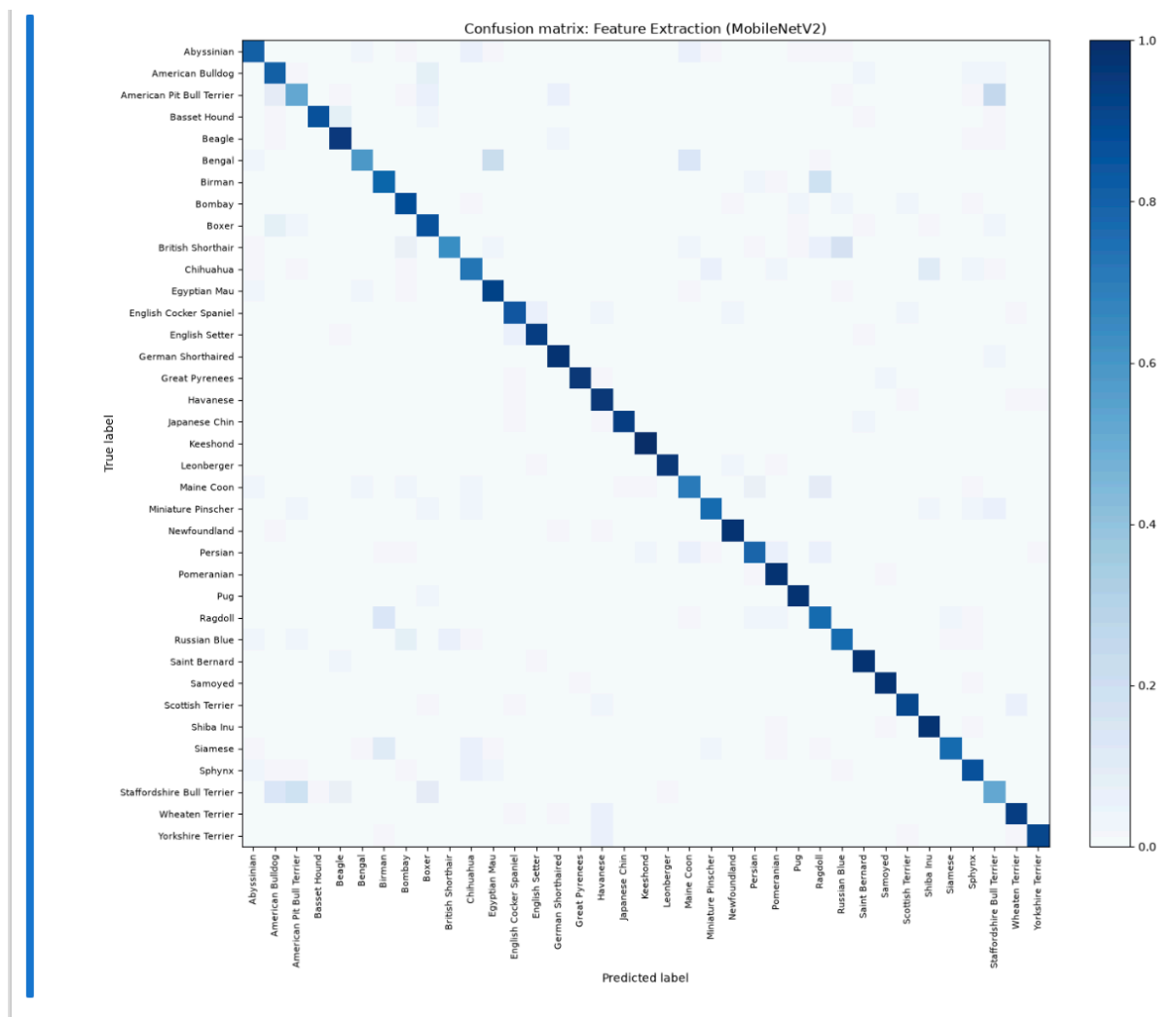


Рисунок 18 — Матрица ошибок (нормированная по строкам) для стратегии Feature Extraction

На тестовой выборке получены accuracy = 85.12 %, precision = 85.33 %, recall = 85.04 %, F1 = 84.85 % (macro-усреднение). Время обучения 1.5 мин.

Анализ. Уже после первой эпохи точность на валидации превысила 80 %: признаки, выученные на ImageNet, почти без изменений подходят для различения пород. Матрица ошибок имеет ярко выраженную диагональ, неверно классифицировано 546 изображений из 3 669. Ошибки сосредоточены в парах визуально похожих пород: American Pit Bull Terrier → Staffordshire Bull Terrier (24); Bengal → Egyptian Mau (23); Birman → Ragdoll (18); Staffordshire Bull Terrier → American Pit Bull Terrier (16); British Shorthair → Russian Blue (16). Это породы с очень похожим внешним видом: американский питбульер и стаффордширский бульерер почти неотличимы по фото, бенгальская кошка и египетский мау обе пятнистые,

бирманская кошка и рэгдолл обе длинношерстные с колор-пойнт окрасом, британская короткошерстная и русская голубая обе серые. Ошибок «кошка ↔ собака» всего 72: грубые различия признаки ImageNet передают хорошо, а тонкие отличия между похожими породами им даются хуже, так как сеть не обучалась выделять именно их.

2.4 Стратегия Fine-tuning

В качестве начального приближения взята модель после Feature Extraction: новый выходной слой уже обучен, поэтому в начале дообучения градиенты не будут большими и случайными и не разрушат предобученные веса. Разморожены верхние блоки базовой сети features[14:] (последние 5 из 19 блоков). Именно они отвечают за наиболее специфичные, высокоуровневые признаки (части тела, формы морды, фактура шерсти), тогда как нижние блоки (края, текстуры) универсальны и остаются замороженными. Слои BatchNorm в замороженных блоках переведены в режим eval, чтобы не изменялась накопленная статистика. Число обучаемых параметров выросло до 1 728 741 (76.1 % всех весов): верхние блоки MobileNetV2 самые «широкие» (до 320 каналов в bottleneck и 1280 в последней свертке), поэтому в них сосредоточена большая часть параметров сети.

Используется дифференцированный низкий темп обучения: 10^{-4} для размороженных блоков и 10^{-3} для классификатора, с косинусным уменьшением (CosineAnnealingLR) в течение 6 эпох. Маленький learning rate необходим, чтобы лишь немного скорректировать полезные предобученные веса, а не переобучить их заново на небольшом наборе данных (иначе возникает переобучение и «катастрофическое забывание»). Результаты приведены на рисунках 19–21.

```
[9]: ft_model = copy.deepcopy(fe_model)
UNFREEZE_FROM = 14
for i, block in enumerate(ft_model.features):
    for p in block.parameters():
        p.requires_grad = i >= UNFREEZE_FROM

# BatchNorm в замороженных блоках оставляем в режиме eval, чтобы не сбивать их статистику
def freeze_bn(model):
    for i, block in enumerate(model.features):
        if i < UNFREEZE_FROM:
            for mod in block.modules():
                if isinstance(mod, nn.BatchNorm2d): mod.eval()

total, trainable = count_params(ft_model)
print(f'Обучаемых параметров: {trainable:,} из {total:,} ({100*trainable/total:.2f}%)')

EPOCHS_FT = 6
opt_ft = torch.optim.Adam([
    {'params': [p for p in ft_model.features.parameters() if p.requires_grad], 'lr': 1e-4},
    {'params': ft_model.classifier.parameters(), 'lr': 1e-3},
])
sched_ft = torch.optim.lr_scheduler.CosineAnnealingLR(opt_ft, T_max=EPOCHS_FT)

_orig_train = ft_model.train
def _train(mode=True):
    _orig_train(mode)
    if mode: freeze_bn(ft_model)
    return ft_model
ft_model.train = _train

t0 = time.time()
hist_ft = fit(ft_model, opt_ft, EPOCHS_FT, scheduler=sched_ft, tag='FT')
time_ft = time.time() - t0
print(f'Время обучения: {time_ft/60:.1f} мин')

Обучаемых параметров: 1,728,741 из 2,271,269 (76.11%)
[FT] epoch 1/6  train loss 0.375 acc 0.899 | val loss 0.341 acc 0.872 | 16s
[FT] epoch 2/6  train loss 0.219 acc 0.934 | val loss 0.291 acc 0.886 | 15s
[FT] epoch 3/6  train loss 0.165 acc 0.956 | val loss 0.255 acc 0.918 | 15s
[FT] epoch 4/6  train loss 0.124 acc 0.970 | val loss 0.252 acc 0.891 | 15s
[FT] epoch 5/6  train loss 0.100 acc 0.980 | val loss 0.249 acc 0.913 | 15s
[FT] epoch 6/6  train loss 0.089 acc 0.984 | val loss 0.245 acc 0.910 | 15s
Время обучения: 1.5 мин
```

Рисунок 19 — Процесс обучения в режиме Fine-tuning

```
[10... _, y_pred_ft = predict(ft_model, test_loader)
m_ft = metrics(y_true, y_pred_ft)
print('Fine-tuning, метрики на тестовой выборке:')
for k, v in m_ft.items(): print(f' {k:10s} = {100*v:.2f}%',)
print('\nСамые частые ошибки:'); top_confusions(y_true, y_pred_ft)

Fine-tuning, метрики на тестовой выборке:
Accuracy   = 88.01%
Precision   = 88.16%
Recall      = 87.91%
F1          = 87.85%

Самые частые ошибки:
Staffordshire Bull Terrier  -> American Pit Bull Terrier : 19 ошибок
Birman                     -> Ragdoll : 18 ошибок
American Pit Bull Terrier   -> Staffordshire Bull Terrier : 17 ошибок
American Pit Bull Terrier   -> American Bulldog : 15 ошибок
Staffordshire Bull Terrier  -> American Bulldog : 13 ошибок

[11... plot_confusion(y_true, y_pred_ft, 'Confusion matrix: Fine-tuning (MobileNetV2)')
```

Рисунок 20 — Метрики модели Fine-tuning на тестовой выборке

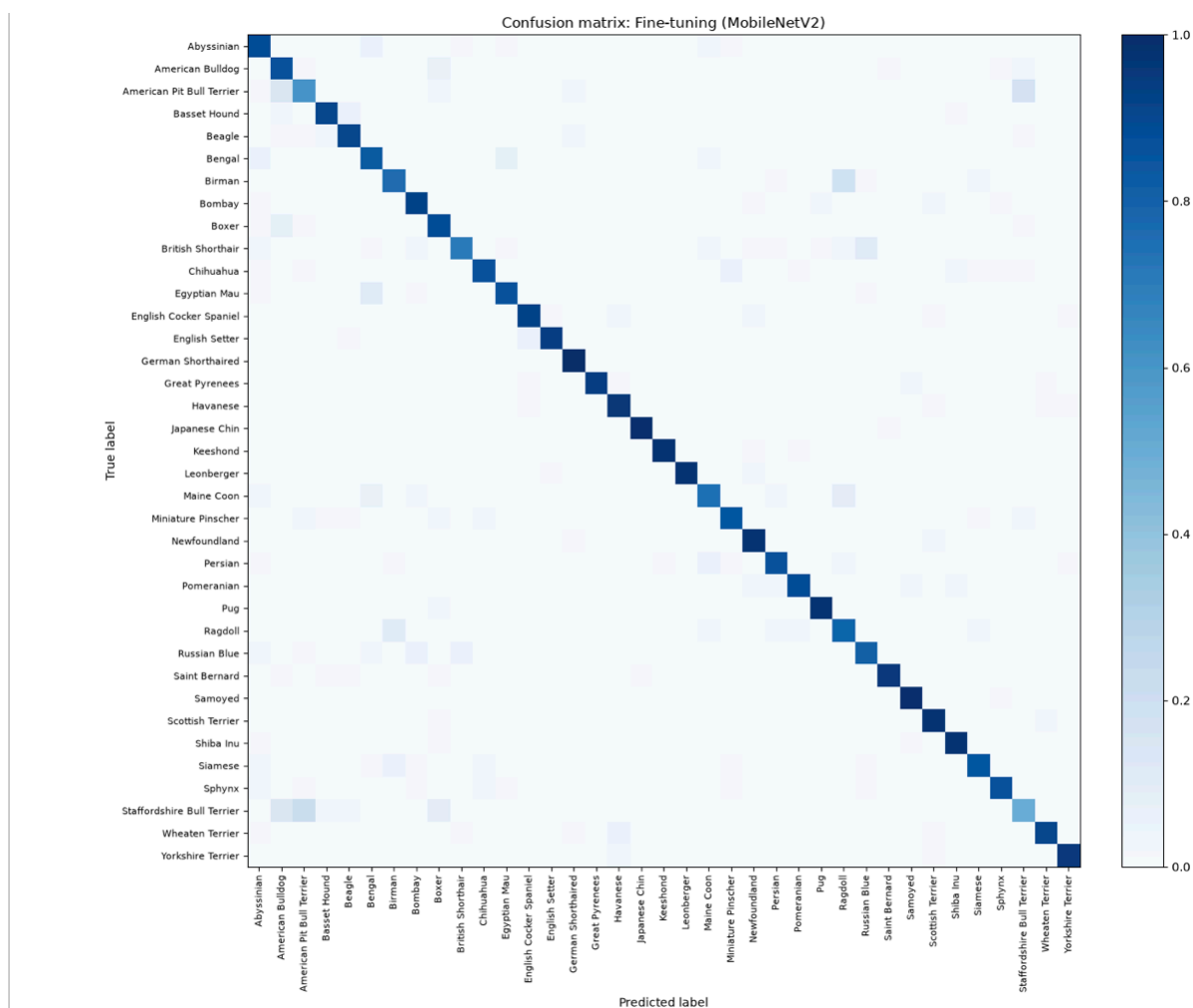


Рисунок 21 — Матрица ошибок (нормированная по строкам) для стратегии Fine-tuning

На тестовой выборке получены accuracy = 88.01 %, precision = 88.16 %, recall = 87.91 %, F1 = 87.85 %. Время дообучения 1.5 мин.

2.5 Сравнение стратегий

Таблица 7 — Сравнение стратегий на тестовой выборке (3 669 изображений, макро-усреднение)

Метрика	Feature Extraction	Fine-tuning	Разница
Accuracy	85.12 %	88.01 %	+2.89 п.п.
Precision	85.33 %	88.16 %	+2.83 п.п.
Recall	85.04 %	87.91 %	+2.87 п.п.
F1	84.85 %	87.85 %	+3.00 п.п.
Время обучения, мин	1.5	1.5 (+ FE)	

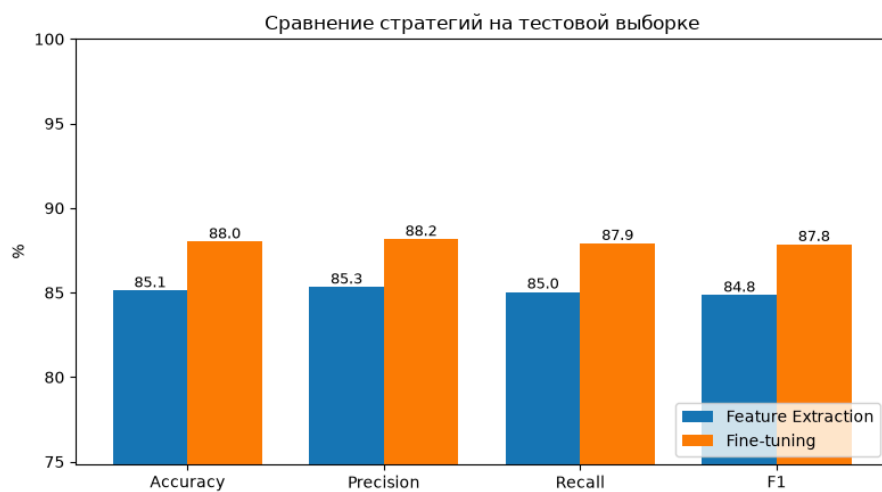


Рисунок 22 — Сравнение метрик двух стратегий

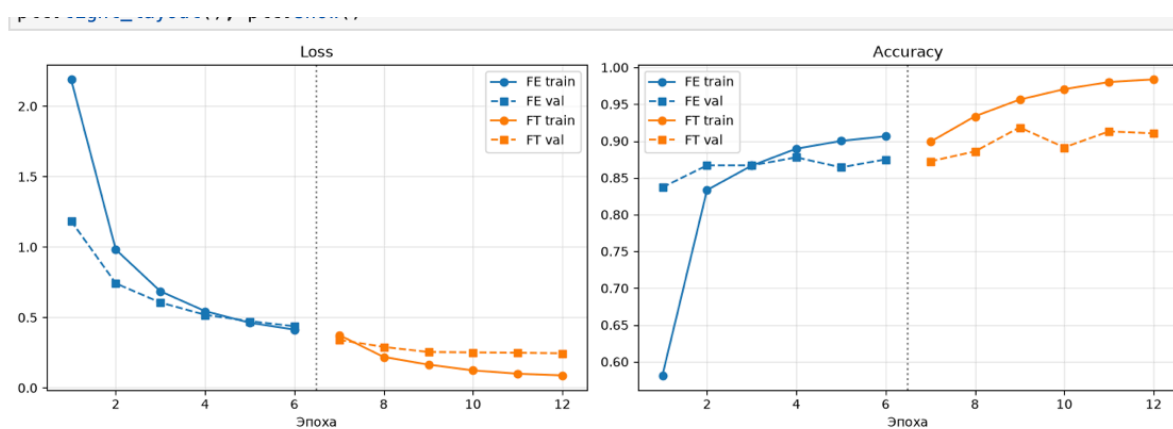


Рисунок 23 — Кривые обучения: Feature Extraction (эпохи 1–6) и Fine-tuning (эпохи 7–12)

```
[15... # Примеры предсказаний модели после Fine-tuning (зелёный: верно, красный: ошибка)
fig, axes = plt.subplots(3, 6, figsize=(16, 8.5))
for ax, i in zip(axes.flat, np.random.RandomState(7).choice(len(test_set), 18, replace=False)):
    img, label = test_set[i]
    ax.imshow(denorm(img)); ax.axis('off')
    p = y_pred_ft[i]
    ax.set_title(f'{class_names[p]}\n(true: {class_names[label]})', fontsize=8,
                color='green' if p == label else 'red')
plt.tight_layout(); plt.show()
```

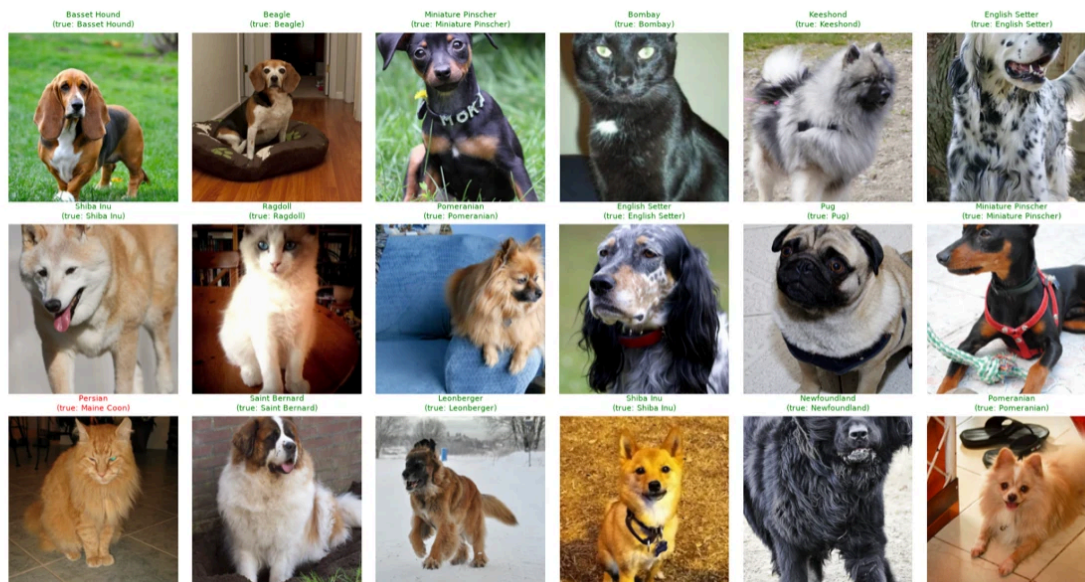


Рисунок 24 — Примеры предсказаний (зеленый: верно, красный: ошибка)

Анализ. Fine-tuning улучшил все метрики примерно на 2.9 процентных пункта: accuracy 85.12 % → 88.01 %, precision 85.33 % → 88.16 %, recall 85.04 % → 87.91 %. Число ошибок на тесте сократилось с 546 до 440, то есть на 19 %, а ошибок «кошка ↔ собака» стало 37 вместо 72. Precision и recall близки друг к другу в обоих случаях: классы сбалансированы, и модель не отдает предпочтения отдельным породам.

Кривые обучения показывают, почему дообучение дает выигрыш. В режиме Feature Extraction к последней эпохе точность на валидации вышла на плато (~88 %): возможности линейного классификатора над фиксированными признаками исчерпаны. После разморозки верхних блоков loss снова пошел вниз, валидационная точность выросла до ~92 %: сеть подстроила высокоуровневые признаки под различия пород (форма морды, уши, окрас). При этом точность на обучающей выборке (98 %) заметно выше валидационной, то есть появились признаки переобучения. Именно поэтому важно использовать маленький learning rate, небольшое число эпох, аугментацию и размораживать только верхние блоки.

Самыми сложными классами остались Staffordshire Bull Terrier ($F1 = 0.55$), American Pit Bull Terrier ($F1 = 0.64$), Ragdoll ($F1 = 0.74$): эти породы путаются между собой даже у людей. Лучше всего распознаются породы с уникальной внешностью: Japanese Chin ($F1 = 0.99$), Keeshond ($F1 = 0.98$), Leonberger ($F1 = 0.98$).

Вывод о целесообразности дообучения. Для данной задачи Fine-tuning целесообразен: при таком же числе дополнительных эпох и времени (1.5 мин) он уменьшает число ошибок на 19 %. Задача мелкозернистая, и признаки ImageNet хоть и близки к ней, но не оптимальны для различения похожих пород, поэтому подстройка верхних слоев дает выигрыш. Feature Extraction стоит использовать как первый этап (быстро, без риска переобучения, 2 % обучаемых параметров) и как самостоятельное решение, когда данных очень мало или ограничены вычислительные ресурсы. Если выборка большая и сильно отличается от ImageNet (например, медицинские или спутниковые снимки), выигрыш от Fine-tuning будет еще больше.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были получены практические навыки применения методов обучения с учителем для задач классификации и регрессии, а также использования предобученных нейросетей.

В первой части с помощью `scikit-learn` выполнена предобработка данных и кодирование меток, построены логистический, наивный байесовский и SVM-классификаторы, линейные, полиномиальный и SVM-регрессоры. Показано влияние параметра регуляризации C на границы классов, важность масштабирования признаков для SVM, преимущество нелинейного RBF-ядра, а также опасность переобучения и численной неустойчивости полиномиальной модели высокой степени: полином 10-й степени оказался хуже линейной регрессии и на контрольной точке, и на тестовой выборке. Матрица неточностей позволила определить, какие именно классы путает классификатор.

Во второй части решена задача классификации 37 пород кошек и собак (Oxford-IIIT Pet) с помощью предобученной сети MobileNetV2. Стратегия Feature Extraction (обучение только нового выходного слоя) за 6 эпох дала accuracy 85.12 %. Стратегия Fine-tuning (разморозка верхних 5 блоков, learning rate 10^{-4}) повысила accuracy до 88.01 %, precision до 88.16 %, recall до 87.91 %. Основные ошибки связаны с визуально похожими породами. Transfer learning позволил получить высокое качество на небольшом наборе данных (около 100 обучающих изображений на класс) за несколько минут обучения на ноутбуке, что при обучении сети с нуля было бы невозможно.

Все поставленные задачи выполнены.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Царёв Д. Современные методы искусственного интеллекта. Лекция 1. История ИИ [Электронный ресурс]. — СПб.: Университет ИТМО, 2026.
2. Лабораторная работа 1. Классификация изображений с применением Transfer Learning: методические указания. — СПб.: Университет ИТМО, 2026. — 22 с.
3. scikit-learn: Machine Learning in Python. User Guide [Электронный ресурс]. — URL: https://scikit-learn.org/stable/user_guide.html (дата обращения: 30.09.2026).
4. PyTorch Documentation. Transfer Learning for Computer Vision Tutorial [Электронный ресурс]. — URL: https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html (дата обращения: 30.09.2026).
5. Parkhi O. M., Vedaldi A., Zisserman A., Jawahar C. V. Cats and Dogs // IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — 2012. — P. 3498–3505.
6. Sandler M., Howard A., Zhu M., Zhmoginov A., Chen L.-C. MobileNetV2: Inverted Residuals and Linear Bottlenecks // IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). — 2018. — P. 4510–4520.
7. Yosinski J., Clune J., Bengio Y., Lipson H. How transferable are features in deep neural networks? // Advances in Neural Information Processing Systems (NeurIPS). — 2014. — Vol. 27.